DigitalOcean Deployment

This deployment path moves the CloudGuard platform off AWS. AWS remains only an optional scan target that users can add through the UI.

Target Architecture

- **Compute:** DigitalOcean Droplet running Docker Compose
- **Database:** PostgreSQL container for the starter setup, or DigitalOcean Managed PostgreSQL for production
- **Queue:** PostgreSQL-backed `scan_jobs`
- **Worker:** `scan-worker` container polling CloudGuard jobs
- **Evidence Storage:** DigitalOcean Spaces through the S3-compatible object storage settings
- **TLS / Routing:** Dokploy/Traefik or another external reverse proxy. This compose file no longer starts Caddy or binds ports `80/443`.
- **AI:** Optional OpenAI-compatible endpoint, including self-hosted models on Proxmox/E2E/DigitalOcean

1. Create DigitalOcean Resources

Create:

- One Ubuntu Droplet, 4 GB RAM minimum for light scans, 8 GB+ recommended
- One DigitalOcean Space for evidence artifacts
- Optional DigitalOcean Managed PostgreSQL if you do not want the DB on the Droplet
- DNS `A` record pointing your domain to the Droplet IP

Automated Terraform Path

From the repo root:

```
cd deploy/digitalocean/terraform
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars
terraform init
terraform apply
terraform output -raw ssh_target
```

This creates:

- Droplet
- Firewall for SSH, plus HTTP/HTTPS only if a reverse proxy such as Dokploy/Traefik is installed on the VM
- Private DigitalOcean Space for evidence
- Project grouping
- Deploy user with Docker installed by cloud-init

Then return to the repo root and create the runtime env:

```
cd ../../..
DOMAIN=cloudguard.example.com \
OBJECT_STORAGE_BUCKET="$(cd deploy/digitalocean/terraform && terraform output -raw space
OBJECT_STORAGE_ACCESS_KEY_ID=replace-with-spaces-key \
OBJECT_STORAGE_SECRET_ACCESS_KEY=replace-with-spaces-secret \
./deploy/digitalocean/generate-env.sh
```

Deploy the local repo to the Droplet:

```
./deploy/digitalocean/deploy-to-droplet.sh \
  "$(cd deploy/digitalocean/terraform && terraform output -raw ssh_target)" \
  .env.digitalocean.local
```

2. Prepare The Droplet

Skip this section if you used the Terraform path above.

```
sudo apt update
sudo apt install -y ca-certificates curl git
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
```

Clone the repo:

```
git clone <repo-url> Cloud_Vulnerabilities
cd Cloud_Vulnerabilities
```

Create env:

```
cp .env.digitalocean.example .env
nano .env
```

Important values:

```
DOMAIN=cloudguard.example.com
BACKEND_BIND_ADDRESS=127.0.0.1
BACKEND_PORT=8000
POSTGRES_PASSWORD=strong-password
WORKER_TOKEN=strong-worker-token

OBJECT_STORAGE_ENDPOINT_URL=https://blr1.digitaloceanspaces.com
OBJECT_STORAGE_REGION=blr1
OBJECT_STORAGE_BUCKET=cloudguard-evidence
OBJECT_STORAGE_ACCESS_KEY_ID=...
OBJECT_STORAGE_SECRET_ACCESS_KEY=...

SANDBOX_LABS_ENABLED=true
```

```
SANDBOX_IAC_DEPLOY=true
SANDBOX_AWS_DEPLOY=false
SANDBOX_GCP_DEPLOY=false
SANDBOX_KUBERNETES_DEPLOY=false
```

For a Proxmox/self-hosted AI model:

```
AI_BASE_URL=http://your-proxmox-model-host:11434/v1
AI_MODEL=llama3.1
AI_API_KEY=local-model
```

3. Start CloudGuard

```
docker compose -f deploy/digitalocean/docker-compose.yml --env-file .env up -d --build
```

Check:

```
docker compose -f deploy/digitalocean/docker-compose.yml --env-file .env ps
docker compose -f deploy/digitalocean/docker-compose.yml --env-file .env exec backend cu
curl -fsS http://127.0.0.1:8000/api/workers/status
```

For public access, route your domain to the backend on `127.0.0.1:8000` through Dokploy/Traefik, Cloudflared, or another reverse proxy. Do not run a second service on ports `80` and `443` if Dokploy owns them.

Scanner-side services use `restart: always` and heartbeat every 15 seconds. In the UI, open `/operations` and confirm Worker Availability shows `online`. If it is offline, restart the worker services:

```
docker compose -f deploy/digitalocean/docker-compose.yml --env-file .env up -d --build s
```

4. Managed PostgreSQL Option

If using DigitalOcean Managed PostgreSQL, update `.env`:

```
DATABASE_URL=postgresql://user:password:host:25060/dbname?sslmode=require
```

Then remove or ignore the `postgres` service and point `backend` and `scheduler-worker` at the managed URL.

5. What Is No Longer AWS-Dependent

- App hosting
- Scan job queue
- Worker execution
- Evidence artifact storage
- TLS and routing when handled by Dokploy/Traefik or another non-AWS reverse proxy
- Optional AI inference endpoint

AWS-specific code remains only for scanning customer AWS accounts and generating AWS IAM setup templates.